

Weeks 12 and 13 Project Report

Emerging Topics: Blockchain, DApps, and E-commerce Security

CS 475 Introduction to Computer Security
Maria Nyolcas

May 2026

Introduction

This installment covers Part IV of the continuous project, focused on emerging topics: blockchain and decentralized applications (Lesson 1), and e-commerce security issues (Lesson 2). The work follows the 1-2-1 ponderation model with theory, hands-on lab activity, and critical reflection. All practical activity was performed in a safe local environment using a toy blockchain implementation in Python. No real assets, production wallets, or public mainnet transactions were used.

From a security-engineering perspective, the objective was not only to define concepts but to test concrete security properties under controlled failure conditions. The lab therefore emphasized one measurable question: when state is altered after mining, does validation fail for the expected cryptographic reason and at the expected position in the chain. This report documents both the expected behavior and its limitations.

Part A - Theory and Guided Research

A-1 What a blockchain is and what problem it solves

A blockchain is an append-only ledger where each block references the hash of the previous block. This creates tamper evidence: modifying historical data changes all downstream hashes. Combined with consensus, blockchain reduces dependence on a single trusted administrator by distributing validation among participants. In security terms, it shifts trust from one operator to protocol rules plus economic incentives.

This model addresses integrity and ordering of shared events, but not truthfulness of inputs. If an oracle submits false external data, or if users sign malicious transactions due to phishing, the chain may still remain internally consistent while preserving a harmful state. In other words, blockchain improves who can rewrite history/guarantees, not who can be deceived at input time/guarantees.

A-2 Blockchain versus DApp

A blockchain is a data structure plus consensus mechanism. A DApp is an application architecture that uses that chain as its trust-preserving backend.

Criterion	Blockchain	DApp
What it is	Ledger structure + consensus rule	Application with smart-contract backend
Main security primitive	Hash chaining + distributed validation	Deterministic contract execution + public state
Typical attack focus	Consensus capture, key theft, network partition	Contract bugs, oracle manipulation, front-end phishing
Upgrade difficulty	Forks or validator governance	Proxy patterns, migrations, governance voting

A-3 Vulnerability classes

Three classes that are particularly unique or amplified in smart contracts: (1) reentrancy, where external calls re-enter vulnerable state-changing functions; (2) oracle manipulation, where on-chain logic

trusts attackable off-chain price feeds; (3) irreversible arithmetic and logic mistakes in deployed contract code.

Three classes not unique but harder to fix in this environment: (1) access-control errors (owner/admin role misuse), (2) input-validation bugs, (3) denial-of-service by resource exhaustion. These are known software-security classes, but blockchain immutability and expensive upgrade paths make post-deployment correction difficult.

Practical mitigations commonly recommended in secure development lifecycles include checks-effects-interactions ordering, pull-payment patterns, role-based authorization audits, multi-signature governance for privileged functions, circuit-breaker (pause) controls, and independent code review before deployment. These controls reduce risk but do not fully remove economic attack incentives in adversarial public networks.

Part B - Hands-on Practical Work

B-1 Local toy blockchain implementation

A small Python blockchain was implemented with these elements: transaction objects, block objects, SHA-256 hashing, proof-of-work mining with a fixed prefix target, and full-chain validation. Two valid blocks were mined after a genesis block.

Implementation details used in this lab:

- Each block hash was computed from a canonical serialized payload including index, timestamp, nonce, previous hash, and transactions.
- Proof-of-work required the block hash to begin with a fixed number of leading zeros (difficulty target).
- Validation checked both local block integrity (recomputed hash equals stored hash) and global linkage integrity (`previous_hash` equals prior block hash).

This separation between local and linkage checks is important because it helps classify which control failed first during tamper scenarios.

B-2 Deliberate break and validation failure

The first non-genesis block was tampered by modifying a transaction amount after mining. Validation then failed with a content-hash mismatch. This demonstrates integrity protection from hash chaining: tampering is detectable even without centralized monitoring.

Observed behavior matched expectation: the earliest failing point was the modified block itself, where recomputation no longer matched the stored hash. No ambiguity was observed in the validator output, which is useful for incident triage because the failure location identifies whether corruption started in block content or in chain linkage.

B-3 Repair attempt and security interpretation

An attacker simulation re-mined only the tampered block. Validation still failed because downstream blocks retained stale `previous_hash` values. The practical lesson is that successful history rewriting requires re-mining all subsequent blocks and overtaking honest participants, which raises attack cost significantly in real networks.

The result also highlights a common misconception: proof-of-work does not make tampering impossible, it makes tampering economically and operationally expensive under honest-majority assumptions. If an attacker gains majority validation power, this defense degrades, which is why consensus decentralization is itself a security control.

B-4 E-commerce security matrix

Lesson 2 was mapped to practical e-commerce risks and controls.

Risk class	Business impact	Control baseline
Card-not-present fraud	Chargebacks and direct loss	3-D Secure 2, velocity limits, device fingerprinting
Credential stuffing	Account takeover and fraud orders	MFA, breached-password blocking, adaptive rate limiting

Session hijacking	Unauthorized checkout activity	Secure/HttpOnly/SameSite cookies, short token lifetime
Supply-chain JS compromise	Payment skimming (Magecart style)	CSP, Subresource Integrity, third-party script governance

Part C - Critical Reflection and Syllabus Tie-in

This installment connects directly to earlier course topics.

Authentication protocols and key management. Blockchain ownership is identity by private key control. This resembles authentication but without central reset. Key theft therefore has immediate asset impact and weak recovery options.

Applied cryptography. Hash functions and digital signatures are foundational: hashes secure block linkage; signatures authorize state transitions. This is not abstract cryptography but protocol-level control enforcement.

Threats, attacks, and defenses. Consensus-level threats (majority control, eclipse conditions) mirror network attack themes, while DApp threats map to software vulnerabilities already studied in web security.

From a defender viewpoint, this means controls must be layered across protocol, application, and user endpoint. A chain can be mathematically sound while users are still compromised through wallet malware, social engineering, or poisoned dependencies.

Operating systems, malware, and web security. Many blockchain incidents are not chain breaks but endpoint compromises: clipboard malware, wallet extension phishing, malicious dependencies, and compromised front-ends. This reinforces a key portfolio theme: secure protocols fail if endpoints are weak.

Security models and policy issues. Classical centralized policy models assume a trusted administrator can revoke, patch, and rollback. Decentralized systems reduce administrator trust but also reduce emergency control options. The tradeoff is explicit: stronger censorship resistance versus harder incident recovery.

In policy terms, organizations adopting blockchain components should predefine incident playbooks for key compromise, contract pause criteria, governance voting thresholds, and legal/operational ownership boundaries. Without this preparation, technical decentralization can become operational ambiguity.

Conclusion

Blockchain solves a specific trust problem: shared state integrity without a single operator. It does not remove security engineering responsibilities. DApps inherit blockchain strengths but also expose new failure modes where code immutability and ecosystem complexity increase remediation cost. For e-commerce, decentralized payment paths do not replace core controls such as strong authentication, anti-fraud analytics, secure session management, and dependency governance. The practical lab confirms the underlying principle: tampering is detectable by construction, but total security still depends on the surrounding system.